

ChurnSense — Simple Project Overview

What problem does this solve?

Telecom companies (internet/phone providers) lose customers over time — this is called **churn**. The problem is that a company usually only finds out a customer left *after* they've already cancelled, which is too late to do anything about it.

ChurnSense's idea: build a machine-learning model that looks at a customer's current information (contract type, payment method, which services they have, how long they've been a customer...) and predicts whether that customer is likely to churn *before* it happens. If a company knows this early, it can reach out and try to retain that customer (special offer, better support, etc.).

The project has 3 main parts

1) The Machine Learning model

It was trained on a real dataset of over 7,000 telecom customers (IBM's Telco Customer Churn dataset). Three models were tried and compared (Logistic Regression, Random Forest, Gradient Boosting), and **Logistic Regression** was chosen because it gave the best overall balance of results. The trained model is saved to a file (`churn_model.pkl`) so the website can reuse it instantly without retraining every time.

2) The Backend (Python / Flask)

This is the "brain" that runs the website:

- Receives customer data from the website.
- Prepares it exactly the way the training data was prepared (encoding, column order, etc.).
- Feeds it to the saved model and gets back a churn probability (a number between 0 and 1).
- Compares that probability to a chosen threshold (0.40) and decides: "Yes, likely to churn" or "No."
- Manages user accounts (register, login, logout) and permissions (regular user, analyst, manager, admin).

3) The Frontend (HTML / CSS / JavaScript)

This is what the user sees in the browser:

- **A public landing page:** explains the project and lets any visitor try a simple version of the prediction form without logging in.
- **A login / register page.**
- **A dashboard** for logged-in users: analyze customers, view past prediction history, see statistics, use a "what-if" Risk Simulator, and (for admins) manage other users' roles.

- The site supports two languages, **English and Arabic**, and switches text direction (left-to-right / right-to-left) automatically.

Where is the data stored?

Supabase (a cloud database service built on PostgreSQL) stores:

- User accounts and their roles.
- Each user's preferences (language, dark mode, etc.).
- Every saved prediction (when it happened, the result, the probability).

The database is protected using **Row Level Security (RLS)** — every user can only ever see their own data, even if someone tries to tamper with requests from the browser.

How does one customer analysis flow through the system?

1. The user opens the "Analyze a customer" form and fills in the customer's information (about 24 fields: gender, contract, internet type, charges, etc.).
2. The website sends this data to the Flask server using a `fetch` request.
3. Flask prepares the data and passes it to the saved model.
4. The model returns a churn probability (say, 0.63 = 63%).
5. Flask compares it to the 0.40 threshold to decide Yes/No, and saves the result to the database (if the user is logged in).
6. The website displays the result clearly (a progress bar, red for higher risk, teal/green for lower risk).

Roles inside the system

Role	What they can do
User	Analyze customers, manage their own profile
Analyst	+ view their full prediction history, analytics, and the "what-if" simulator
Manager	+ view history and analytics (no simulator)
Admin	Everything, plus the ability to change any other user's role

One-sentence summary

ChurnSense = a **machine-learning model that predicts customer churn** + a **full web application (login, roles, dashboard)** that turns that model into something a real team can actually use day to day, instead of it staying a Jupyter notebook on one person's laptop.